

RAVI System

Recognition Automated Via Infrared

Onboarding documentation for the FTIR microplastic identification platform — covering the Flutter client, the Python MLP inference server, and the Firebase realtime database schema.

Project	RAVI — Microplastic Identification by FTIR
Author	Daniel Martins (PhD)
Audience	Undergraduate research student (IC)
Components	Flutter app · FastAPI server · Firebase RTDB
Repository	/home/danielma/Documents/Doutorado/Sistema

Table of Contents

1. System overview
2. Architecture and component map
3. Repository layout
4. Setup and running locally
5. MLP inference server (mlp_server.py)
6. Spectral storage (CSV files)
7. Firebase database schema (multi-tenant)
8. Flutter client architecture
9. User flows
10. Internationalisation (i18n)
11. Common tasks (how-to)
12. Troubleshooting
13. Roadmap and known limitations
- A. Glossary

1. System overview

RAVI is a desktop and web application that supports microplastic research workflows: cataloguing field collections, registering individual samples, importing FTIR spectral data, running automatic polymer identification through a multilayer perceptron (MLP) model, and visualising the spectrum together with the model attention map.

The system is **multi-tenant**: each institution / laboratory has its own isolated namespace inside Firebase. Users of one institution cannot see or modify data from another. Only administrators can register new institutions, which prevents accidental tenant pollution during early adoption.

What it solves

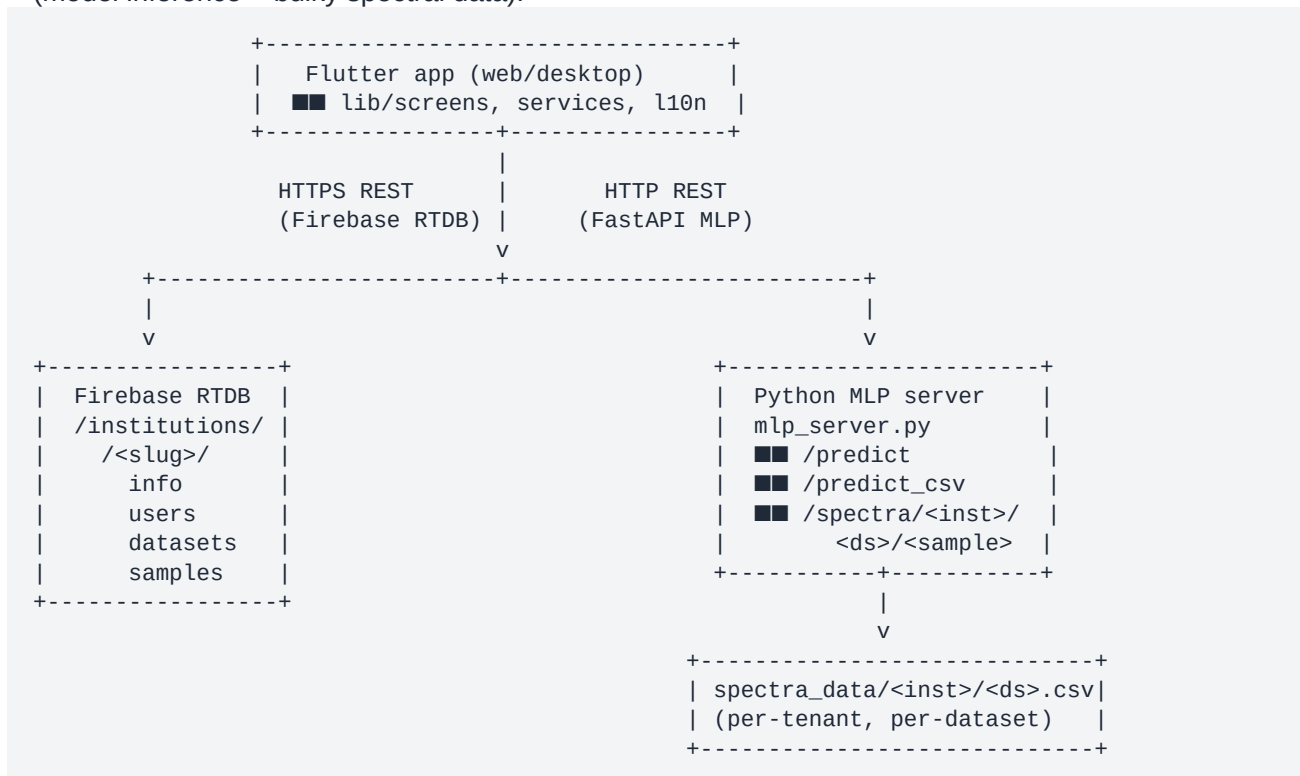
- bullet Centralises FTIR collections across collaborators in a single structured database, replacing scattered spreadsheets.
- bullet Automates polymer identification (PE, PP, PS, PA, EVA, cellulose) by reusing the trained MLP from the BRACIS 2026 paper pipeline.
- bullet Stores the spectral signal and the model's saliency map together, so reviewers can visually justify each identification.
- bullet Provides traceable random sample IDs that do not leak collection order or operator-chosen names from the original CSV.

Out of scope (for now)

- bullet Production-grade authentication: passwords are obfuscated with a salted base64 hash, not bcrypt/argon2. Plan: migrate to Firebase Auth before any external rollout.
- bullet Granular per-collection permissions: every user inside an institution sees every collection of that institution.
- bullet Automated retraining loop: verified samples are stored, but the training script is run manually outside the app.

2. Architecture and component map

Three independent processes cooperate. The Flutter client is the only thing the end user sees. It talks to two backends over HTTP: Firebase Realtime Database (metadata) and the local FastAPI server (model inference + bulky spectral data).



Why two backends?

Metadata (sample names, polymer labels, attention summaries) lives in Firebase: small, indexed, multi-user. The raw spectrum (~1763 intensities per sample) would bloat each Firebase node, so it is stored as plain CSV on the inference server, keyed by sample ID. When the detail screen opens, the client fetches metadata from Firebase and the spectrum from the server, then renders both in the same chart.

Network endpoints by default

Service	URL	Notes
Flutter dev server	<code>http://localhost:<flutter port></code>	Served by <code>`flutter run`</code>
MLP server	<code>http://localhost:8000</code>	Started by <code>`python mlp_server.py`</code>
Firebase RTDB	<code>https://deepmicroplastics-default-rtdb.firebaseio.com</code>	Hardcoded in <code>firebase_service.dart</code>

3. Repository layout

```

Sistema/
■■■ mlp_server.py           # FastAPI inference + spectra storage
■■■ extract_scaler.py       # Builds scaler_params.json from training data
■■■ MLP_best_model.h5       # Trained Keras model (FTIR → polymer)
■■■ scaler_params.json      # MinMaxScaler params used by the server
■■■ DB_10cl_100sp.csv       # Sample CSV used during development
■■■ TDB.csv                 # Test database
■■■ spectra_data/           # Created at runtime (per-institution CSVs)
■   ■■■ <institution_slug>/<dataset_id>.csv
■■■ docs/                   # Output directory for this PDF
■■■ deepmicroplastic/       # Flutter project root
    ■■■ lib/
        ■   ■■■ main.dart           # App entry, locale wiring
        ■   ■■■ l10n/               # Generated localisations + ARB
        ■   ■   ■■■ app_en.arb      # English source
        ■   ■   ■■■ app_pt.arb     # Portuguese source
        ■   ■   ■■■ app_localizations*.dart (gen-l10n output)
        ■   ■■■ models/            # Plain Dart data classes
        ■   ■   ■■■ institution_model.dart
        ■   ■   ■■■ user_model.dart
        ■   ■   ■■■ spectrum_model.dart
        ■   ■■■ services/          # IO, no UI
        ■   ■   ■■■ firebase_service.dart # Raw RTDB REST wrapper
        ■   ■   ■■■ institution_service.dart
        ■   ■   ■■■ auth_service.dart
        ■   ■   ■■■ dataset_service.dart
        ■   ■   ■■■ sample_service.dart
        ■   ■   ■■■ spectral_storage_service.dart
        ■   ■   ■■■ csv_import_service.dart
        ■   ■   ■■■ mlp_service.dart
        ■   ■   ■■■ locale_service.dart
        ■   ■■■ screens/           # One screen = one .dart
        ■   ■   ■■■ login_screen.dart
        ■   ■   ■■■ new_institution_screen.dart
        ■   ■   ■■■ ftir_overview_screen.dart (home)
        ■   ■   ■■■ add_dataset_screen.dart
        ■   ■   ■■■ dataset_detail_screen.dart
        ■   ■   ■■■ add_sample_screen.dart
        ■   ■   ■■■ sample_detail_screen.dart
        ■   ■   ■■■ import_csv_screen.dart
        ■   ■   ■■■ manage_users_screen.dart
        ■   ■   ■■■ manage_institutions_screen.dart
        ■   ■   ■■■ user_form_screen.dart
        ■   ■■■ utils/
        ■   ■   ■■■ id_generator.dart # Random traceable IDs
        ■   ■■■ widgets/
        ■   ■   ■■■ ftir_chart.dart # Custom-painted FTIR chart
        ■   ■   ■■■ language_menu.dart
        ■■■ assets/login_bg.jpg    # Login background image
        ■■■ pubspec.yaml
        ■■■ l10n.yaml

```

Two directories deserve attention. `lib/services/` is where every external IO call lives — never hit Firebase or the MLP server directly from a screen. `lib/l10n/` holds the source-of-truth ARB files; the corresponding Dart files are generated by `flutter gen-l10n` and should not be edited by hand.

4. Setup and running locally

Prerequisites

Python 3.13+ with: tensorflow, fastapi, uvicorn, pandas, numpy, pydantic.

Flutter 3.11+ (channel stable). Verify with `flutter --version`.

At Firebase Realtime Database. The default URL is hardcoded in `lib/services/firebase_service.dart` — change it before deploying outside the lab.

Quick start

```
# 1. Python environment for the MLP server
cd /home/danielma/Documents/Doutorado/Sistema
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn tensorflow pandas numpy pydantic

# 2. Start the MLP server (in its own terminal)
python mlp_server.py
# → "Loading model from MLP_best_model.h5 ..."
# → Uvicorn running on http://0.0.0.0:8000

# 3. Flutter project
cd /home/danielma/Documents/Doutorado/Sistema/deepmicroplastic
flutter pub get
flutter gen-l10n          # Only after editing app*.arb
flutter run -d chrome     # or -d linux / -d macos / -d windows
```

First-run experience

If the Firebase database has no `/institutions` node, the login screen automatically opens the institution-creation flow. The user fills in the institution name, an admin username and password — and after saving, the app boots straight into the home screen as that admin. From there, additional institutions can be added through `Menu → Manage Institutions`.

5. MLP inference server (mlp_server.py)

FastAPI process loaded once at startup. It owns the trained model (`MLP_best_model.h5`), the optional scaler (`scaler_params.json`), and the on-disk CSVs that store raw spectra per dataset. CORS is wide-open — fine for the lab network, tighten before any public deployment.

Endpoints

Method	Path	Body / params	Purpose
GET	/health	—	Returns scaler mode + class list
POST	/predict	{wavenumbers, intensities}	Single-sample inference
POST	/predict_csv	multipart file	Batch inference; one CSV row per sample
POST	/spectra/{inst}/{dataset}	{sample_id, wavenumbers, intensities}	Persist a sample spectrum to disk
GET	/spectra/{inst}/{dataset}/{sample_id}		Reload that spectrum
DELETE	/spectra/{inst}/{dataset}/{sample_id}		Remove a single row from the dataset CSV

Inference pipeline

For each sample the server: (1) interpolates the user's (wavenumber, intensity) pairs onto the model's fixed grid of 1763 points from 3998 to 600 cm^{-1} ; (2) applies the MinMaxScaler if `scaler_params.json` is present, otherwise falls back to per-sample normalisation; (3) runs the Keras model and computes the saliency by $|\text{grad} \times x|$, normalised to [0, 1]. The response carries the predicted class, the confidence, the wavenumbers/intensities and the saliency vector aligned with the wavenumber grid.

Polymer classes

The trained model recognises six classes: `EVA`, `PA`, `PE`, `PP`, `PS`, `cellulose_like`. The Flutter client maps these to its `PolymerType` enum and renders them with consistent colours and full names (see `lib/models/spectrum_model.dart`).

6. Spectral storage (CSV files)

Each dataset has its own CSV file under `spectra_data//.csv`. Columns: the first holds the random sample ID, the rest are the intensities indexed by the wavenumbers (CSV header). When the detail screen opens, the Flutter client requests the row by ID and rebuilds the spectrum on the fly.

```
sample_id,3998.0,3995.7,3993.4, ... ,602.4,600.0
smp-x9p7k2qa3jvw,0.0123,0.0119,0.0115, ... ,0.0341,0.0322
smp-3kk2pqahjabc,0.0421,0.0414,0.0407, ... ,0.0091,0.0088
```

Why a CSV per dataset?

- **Flat** RTDB has practical limits on node size — embedding 1763 floats per sample would slow every list query.
- **Plain** CSV is exportable for retraining without ETL: each file is already a valid input for the training pipeline.
- **Tenant** separation is enforced at the filesystem level: the directory layout makes it impossible to read another institution's spectrum by accident.

Re-aligning headers

If a sample comes from a CSV with a slightly different wavenumber grid (e.g. 1700 columns instead of 1763), `save_spectrum` interpolates the new intensities onto the dataset's existing header before appending. This guarantees the file stays rectangular and importable into pandas.

7. Firebase database schema (multi-tenant)

Every entity lives under `/institutions/`, so removing a tenant is a single recursive delete. The Flutter services already enforce this layout — no screen ever builds a raw Firebase path.

```

/institutions
  /<slug>                                # e.g. "usp-iag" (lowercase, ASCII)
    /info
      name:          "USP – IAG"
      createdBy:     "admin"
      createdAt:     1714588800000
    /users
      /<username>                          # e.g. "jsilva"
        name:        "João Silva"
        email:       "jsilva@usp.br"
        department:  "Lab. Oceanografia"
        role:        "admin" | "researcher"
        passwordHash: "<base64('raviDeepMp_' + password)>"
        createdAt:   1714588800000
      /datasets
        /<datasetId>                      # e.g. "ds-1714588900000"
          name:        "Praia do Futuro – Apr/2024"
          description: "Surface sediment, 5 points"
          location:    "Fortaleza, CE"
          createdAt:   1714588900000
          createdBy:   "jsilva"
          microscopeMode: "atr" | "transmission" | "reflection"
          microscopeModel: "Bruker Vertex 70 + Hyperion 3000"
          resolution:   4.0
          numScans:     64
          detectorType: "MCT"
          crystalType:  "Diamante"
          dataType:     "absorbance" | "transmittance"
        /samples
          /<sampleId>                     # random, e.g. "smp-x9p7k2qa3jvw"
            datasetId: "ds-1714588900000"
            name:      "PF-K7XN3F"        (display name, random)
            collectionSite: "Point 4 – North Shore"
            collectionDate: 1714588800000
            dataType:     "absorbance"
            notes:        "..."
            isVerified:   true
            verifiedBy:   "jsilva"
            createdAt:    1714588950000
            createdBy:    "jsilva"
            result:       # optional, set after MLP
              polymer:    "pe"
              confidence: 0.97
              decisionWavenumber: 2919.0
              reasoning:  "..."
              keyPeaks:   ["2919 cm-1 – ...", ...]
              attentionMap:
                - { wn: 3998.0, att: 0.001 }
                - { wn: 3995.7, att: 0.003 }
                ...

```

Where this is implemented

Path prefix	Service file
<code>/institutions</code>	<code>lib/services/institution_service.dart</code>

<code>/institutions/<slug>/users</code>	<code>lib/services/auth_service.dart</code>
<code>/institutions/<slug>/datasets</code>	<code>lib/services/dataset_service.dart</code>
<code>/institutions/<slug>/samples</code>	<code>lib/services/sample_service.dart</code>

Identifier conventions

slug: ASCII lowercase, hyphen-separated. Generated automatically by `InstitutionModel.slugify(name)`.

datasetId: ds-, generated by `DatasetService.save`.

sampleId: smp-<12 random chars>, generated by `SampleIdGenerator.generateInternalId()`.

sample.name: -<6 random chars>, e.g. PF-K7XN3F. The prefix comes from the dataset name (first letters of the first significant words).

8. Flutter client architecture

There is no third-party state-management library: the app relies on `StatefulWidget` plus a single global `ChangeNotifier` for the locale. Each screen owns its own state and calls services directly. This keeps the navigation predictable and easy to reason about.

Layered structure

- `models/` — plain data classes with `toMap` / `fromMap` converters. No framework imports.
- `services/` — every HTTP call lives here. Returns models or null/booleans on failure. UI never imports `http` directly.
- `screens/` — one widget per screen. Reads from services in `initState`, refreshes on `pop`.
- `widgets/` — reusable visual components (`FtirChart`, `LanguageMenu`).
- `utils/` — pure functions (id generator).

Why no Provider / Riverpod / Bloc?

The app does not yet have a workflow that needs cross-screen reactive state. Every navigation is push/pop; data is reloaded from Firebase on return so the UI never goes stale. If the user base grows, switching to Riverpod is the cleanest upgrade path.

Custom-painted spectrum chart

`lib/widgets/ftir_chart.dart` draws the FTIR plot using `CustomPainter`. It overlays three layers: the intensity curve, the model's attention heatmap, and the decision-point dashed line. A toggle lets the user switch between absorbance and transmittance — derived on the fly via the `SpectrumSample.asTransmittance` / `asAbsorbance` getters.

9. User flows

First execution (database empty)

Cold start (empty database):

- Login screen detects no institutions → "Create first admin" banner
- New Institution screen
- User fills institution name + admin credentials
- Returns to login pre-filled with the slug
- Sign in

Returning user

Returning user:

- Login screen → enter institution slug
- Continue → if found, ask username+password
- Sign in → home (FtirOverviewScreen)

Adding samples (three ways)

Creating samples (from a collection):

- Home → tap a collection → DatasetDetailScreen
- "+" New Sample
 - Tab "Individual": fill site/date, optionally attach a single-row CSV
 - save → goes through MLP if attached
 - Tab "Batch": type quantity → save N empty samples,
OR click "Import batch from CSV"
 - ImportCsvScreen with shared site/notes inherited
 - bulk MLP inference, then mass save

Running identification

Running identification later:

- Sample detail → biotech icon (top-right)
- POST /predict to mlp_server
- Saves polymer + attentionMap to Firebase
- Spectrum chart updates with attention heatmap

10. Internationalisation (i18n)

The app starts in English. Users can switch to Portuguese from the language menu, available both on the login screen and inside the home menu. The choice is persisted in `SharedPreferences` by `LocaleService`, which exposes a `ChangeNotifier` that `RaviApp` listens to — switching the locale rebuilds the entire widget tree without restarting.

Adding a new translatable string

```
# 1. Add the new key to BOTH ARB files
#    (lib/l10n/app_en.arb and lib/l10n/app_pt.arb)
"helloUser": "Hello, {name}",
"@helloUser": { "placeholders": { "name": {} } }

# 2. Regenerate Dart files
flutter gen-l10n

# 3. Use it in any widget
final l = AppLocalizations.of(context);
Text(l.helloUser('Daniel'))
```

ARB conventions

bullet Keys use camelCase, grouped by screen/feature (`homeKpiTotalSamples`, `addSampleSiteHint`).

bullet Always edit both `app_en.arb` and `app_pt.arb` — missing keys throw at runtime.

bullet Use `@key` metadata for placeholders, e.g. `"@datasetSamples": { "placeholders": { "n": { "type": "int" } } }`.

bullet Generated files (`app_localizations*.dart`) are committed for editor support but are regenerated on every `flutter gen-l10n` — never edit them by hand.

RAVI acronym

The acronym `RAVI` is fixed across locales. Only the expansion changes: in English it stands for `Recognition Automated Via Infrared`, in Portuguese `Reconhecimento Automatizado Via Infravermelho`.

11. Common tasks (how-to)

Add a new screen

```
// 1. Create lib/screens/my_screen.dart
class MyScreen extends StatelessWidget { ... }

// 2. Add localisations (both ARB) and run flutter gen-l10n.

// 3. Navigate from wherever:
Navigator.push(context, MaterialPageRoute(
  builder: (_) => MyScreen(loggedUser: widget.loggedUser),
));
```

Add a new server endpoint

```
# 1. Add the route in mlp_server.py
@app.post("/something/{institution_slug}")
def my_route(institution_slug: str, req: MyRequest):
    ...

# 2. Add the Dart wrapper in services/
class SomethingService {
  static Future<...> call({...}) async {
    final res = await http.post(...);
    ...
  }
}

# 3. Restart the Python server (Uvicorn auto-reload is OFF by default).
```

Add a new polymer class

```
# 1. Retrain MLP_best_model.h5 with the extra class.
# 2. Update CLASS_NAMES in mlp_server.py.
# 3. Add the matching enum value in lib/models/spectrum_model.dart
#   (PolymerType, label, fullName, color).
# 4. Update _mapPolymer() in csv_import_service.dart and mlp_service.dart.
# 5. Optionally: add diagnostic peaks in the same files.
```

Change the Firebase database URL

Edit the constant `_base` in `lib/services/firebase_service.dart`. This is the single point of change — every other service derives its path from there.

Change the MLP server URL

There are three references to `http://localhost:8000`: `MlpService`, `CsvImportService` and `SpectralStorageService`. When deploying outside the dev box, extract these into a shared constant.

12. Troubleshooting

Symptom	Likely cause	Fix
“Could not reach the MLP server”	mlp_server.py not running, or wrong host	Run python mlp_server.py; verify /health returns 200
Login: Institution not found	Database empty or wrong slug	Use first-run flow, or check /institutions in Firebase
FTIR chart blank after restart	Spectrum CSV missing on the server	Check spectra_data/<inst>/<dataset>.csv exists
CSV import fails 400	CSV without numeric column headers	Make sure column names are wavenumbers (e.g. 600.0)
UI still in Portuguese after change	SharedPreferences cached or hot reload skipped	Restart the app; LocaleService re-reads on boot
flutter gen-l10n complains	Missing key in one of the ARBs	Diff app_en.arb and app_pt.arb — every key must exist in both

13. Roadmap and known limitations

Short term

- bullet Migrate authentication from the current base64 hash to Firebase Authentication. The Dart layer already isolates auth in `AuthService`, so the swap is local.
- bullet Replace the hardcoded `http://localhost:8000` in the three services with a build-time configurable constant.
- bullet Add per-collection ACLs so a researcher only sees collections they participate in.

Medium term

- bullet Hook a retraining pipeline to the verified samples: every time an admin marks a batch as `isVerified`, append those rows to the training set.
- bullet Migrate the dataset / sample listings to Firestore (paginated queries) once the per-tenant data grows past a few thousand rows.
- bullet Move FTIR chart rendering to GPU (Skia paint primitives) for smoother scrolling on long spectra.

Open questions

- bullet Should institutions support sub-tenants (departments)? Today all users of a slug share everything.
- bullet Storage strategy for very large CSVs (>10 MB) — keep on disk or move to object storage?
- bullet Whether to expose a RAVI public API (e.g. for other research groups to submit spectra for blind testing).

Appendix A. Glossary

Term	Meaning
FTIR	Fourier-Transform InfraRed spectroscopy — the technique used to record the absorbance/transmittance s
MLP	Multilayer Perceptron — feed-forward neural network used here to classify the polymer.
Saliency / attention	Per-wavenumber importance derived from the model gradient, $ \text{grad} \times x $. Visualised as the orange heatm
Tenant / institution	An isolated namespace in the database. Identified by a slug.
Slug	ASCII-only, lowercase, hyphenated identifier derived from the institution name.
ARB	Application Resource Bundle — JSON-based file format used by Flutter for translations.
Decision wavenumber	The single wavenumber that the model uses as the strongest evidence for the predicted polymer class. D